

UNIVERSIDADE DE CUIABÁ (UNIC)

VICTOR BELLÉ DALLA NORA

**CHATBOT DE ARTIGOS CIENTÍFICOS: UMA IMPLEMENTAÇÃO LOCAL COM OLLAMA E
MEMÓRIA DE CURTO PRAZO**

CUIABÁ 2025

VICTOR BELLÉ DALLA NORA

**CHATBOT DE ARTIGOS CIENTÍFICOS: UMA IMPLEMENTAÇÃO LOCAL COM OLLAMA E
MEMÓRIA DE CURTO PRAZO**

Trabalho acadêmico apresentado à disciplina de Inteligência
Artificial do curso de Engenharia da Computação da UNIC
(Universidade de Cuiabá).

Professor: Felipe Douglas

CUIABÁ 2025

RESUMO

Este trabalho documenta o desenvolvimento de um protótipo de chatbot de interface de linha de comando (CLI) projetado para auxiliar usuários com perguntas conceituais sobre artigos científicos. O objetivo principal foi construir uma solução 100% local, funcional sem chaves de API ou dependência de serviços externos, garantindo privacidade e custo zero. O sistema utiliza a linguagem Python e a biblioteca requests para interagir diretamente com o endpoint HTTP do servidor Ollama, que executa um modelo de linguagem (como o Llama 3.1) localmente. Para manter a coerência do diálogo, foi implementada uma classe de memória de curto prazo (ShortMemoryChat) que gerencia uma janela deslizante de $k=5$ interações, permitindo que o chatbot retenha o contexto imediato da conversa. O resultado é um assistente didático e funcional que responde a perguntas sobre a estrutura e conceitos de artigos (ex: IMRD), operando inteiramente no hardware do usuário.

Palavras-chave: Chatbot. Ollama. Processamento de Linguagem Natural. API Local. Python.

ABSTRACT

This paper documents the development of a command-line interface (CLI) chatbot prototype designed to assist users with conceptual questions about scientific papers. The main objective was to build a 100% local solution, functional without API keys or dependence on external services, ensuring privacy and zero cost. The system uses the Python language and the `requests` library to interact directly with the HTTP endpoint of the Ollama server, which runs a language model (like Llama 3.1) locally. To maintain dialogue coherence, a short-term memory class (`ShortMemoryChat`) was implemented, managing a sliding window of $k=5$ interactions, allowing the chatbot to retain the immediate conversation context. The result is a functional, didactic assistant that answers questions about the structure and concepts of articles (e.g., IMRD), operating entirely on the user's hardware.

Keywords: Chatbot. Ollama. Natural Language Processing. Local API. Python.

LISTA DE SIGLAS

API – *Application Programming Interface* (Interface de Programação de Aplicações)

CLI – *Command-Line Interface* (Interface de Linha de Comando)

CPU – *Central Processing Unit* (Unidade Central de Processamento)

FIFO – *First-In, First-Out* (Primeiro a Entrar, Primeiro a Sair)

GPU – *Graphics Processing Unit* (Unidade de Processamento Gráfico)

HTTP – *Hypertext Transfer Protocol* (Protocolo de Transferência de Hipertexto)

IMRD – Introdução, Métodos, Resultados e Discussão

JSON – *JavaScript Object Notation*

LLM – *Large Language Model* (Modelo de Linguagem de Grande Escala)

RAG – *Retrieval-Augmented Generation* (Geração Aumentada por Recuperação)

VRAM – *Video Random Access Memory* (Memória de Acesso Aleatório de Vídeo)

LISTA DE QUADROS

Quadro 1 – Diagrama de Fluxo da Aplicação	10
Quadro 2 – Trecho da Classe ShortMemoryChat (Método add)	12
Quadro 3 – Função ollama_chat	13
Quadro 4 – Trecho da Função main	14

SUMÁRIO

1 INTRODUÇÃO	8
2 TECNOLOGIAS E AMBIENTE	9
2.1 LINGUAGEM E DEPENDÊNCIAS	9
2.2 MODELO LOCAL E SERVIDOR OLLAMA	9
2.3 REQUISITOS DE INSTALAÇÃO	9
3 ARQUITETURA E DESIGN	10
3.1 VISÃO GERAL DO FLUXO	10
3.2 DIAGRAMA DE FLUXO	11
3.3 COMPONENTES DE DESIGN	11
4 EXPLICAÇÃO DO CÓDIGO	12
4.1 ARQUIVO PRINCIPAL: chatbot.py	12
4.2 GERENCIAMENTO DA MEMÓRIA	12
4.3 COMUNICAÇÃO HTTP COM OLLAMA	12
4.4 LOOP PRINCIPAL E INTERFACE (CLI)	13
5 INSTRUÇÕES DE EXECUÇÃO	14-15
6 RESULTADOS E DEMONSTRAÇÃO	15
6.1 EXEMPLOS DE INTERAÇÃO	16
6.2 OBSERVAÇÕES DE LATÊNCIA	16
7 LIMITAÇÕES E TRABALHOS FUTUROS	17
7.1 LIMITAÇÕES	17
7.2 TRABALHOS FUTUROS	17
8 CONCLUSÃO	18
REFERÊNCIAS	19

1 INTRODUÇÃO

Modelos de Linguagem de Grande Escala (LLMs) estão redefinindo o acesso à informação. No meio acadêmico, a leitura e compreensão de artigos científicos é uma habilidade fundamental, porém desafiadora. A estrutura densa e o jargão técnico, como o formato IMRD (Introdução, Métodos, Resultados e Discussão), podem ser barreiras para novos pesquisadores. Um chatbot focado em explicar esses conceitos pode servir como uma valiosa ferramenta de auxílio didático.

A maioria das soluções de chatbot de alto desempenho, no entanto, depende de APIs pagas (como OpenAI ou Google) ou de infraestruturas de nuvem complexas. Isso introduz barreiras de custo, dependência de rede e questões de privacidade de dados.

O objetivo central deste trabalho é demonstrar a viabilidade de um chatbot acadêmico funcional, 100% local e sem API, utilizando o Ollama como servidor de modelo. Objetivos secundários incluem:

- Implementar uma comunicação direta com a API HTTP (/api/chat) do Ollama usando Python.
- Gerenciar o estado da conversa através de uma memória de curto prazo.
- Direcionar o comportamento do modelo usando um prompt de sistema focado.

Este é um projeto de escopo deliberadamente limitado, focado na prova de conceito da interação local. O chatbot não implementa RAG (Retrieval-Augmented Generation); ele não lê ou processa arquivos PDF. O foco é a conversa contextual básica, com o modelo respondendo com base em seu conhecimento pré-treinado sobre a estrutura de artigos científicos.

2 TECNOLOGIAS E AMBIENTE

A solução foi desenvolvida com um conjunto de ferramentas focadas em simplicidade e execução local.

2.1 LINGUAGEM E DEPENDÊNCIAS

O projeto foi escrito em Python (versão 3.10+). As dependências externas, gerenciadas por um arquivo `requirements.txt`, são:

- **requests**: Biblioteca padrão para realizar chamadas HTTP (POST) ao servidor Ollama.
- **python-dotenv**: Utilizada para carregar variáveis de ambiente (como o nome do modelo) de um arquivo `.env`, separando configuração de código.
- **colorama**: Biblioteca auxiliar para estilizar a saída do terminal (cores), melhorando a legibilidade da interface de linha de comando (CLI).

2.2 MODELO LOCAL E SERVIDOR OLLAMA

O "cérebro" do chatbot é um modelo de linguagem (ex: llama3.1) executado localmente através do Ollama. O Ollama atua como um servidor leve que gerencia e executa modelos open-source no hardware do usuário (CPU ou GPU).

A comunicação com o modelo não utiliza bibliotecas de orquestração (como LangChain), mas sim chamadas diretas ao endpoint HTTP exposto pelo Ollama, tipicamente `http://localhost:11434`, na rota `/api/chat`.

2.3 REQUISITOS DE INSTALAÇÃO

A execução do projeto requer:

- Um ambiente virtual Python (venv).
- A instalação das dependências (`pip install -r requirements.txt`).
- A instalação prévia do servidor Ollama e o download do modelo desejado (ex: `ollama pull llama3.1`).

3 ARQUITETURA E DESIGN

O fluxo de dados do chatbot é linear e direto, gerenciado por um único script Python.

3.1 VISÃO GERAL DO FLUXO

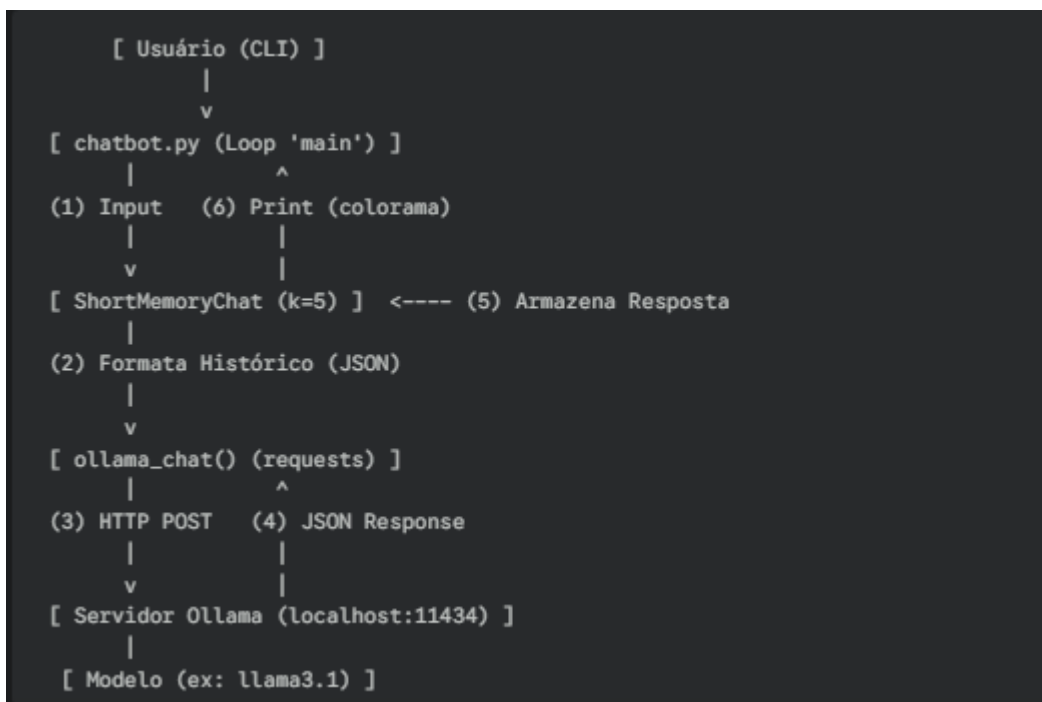
A interação do usuário segue 6 etapas principais:

1. **Entrada do Usuário:** O usuário digita uma pergunta no terminal.
2. **Consulta à Memória:** O script anexa a nova pergunta ao histórico de mensagens mantido pela classe ShortMemoryChat.
3. **Requisição HTTP:** O histórico completo (incluindo o prompt de sistema e a janela de k=5 interações) é enviado via requests.post para o endpoint /api/chat do Ollama.
4. **Processamento Local:** O Ollama recebe a requisição, carrega o modelo e gera uma resposta completa (pois stream=False).
5. **Resposta HTTP:** O Ollama retorna a resposta completa em um objeto JSON.
6. **Saída no Terminal:** O script extrai o conteúdo da resposta, armazena no histórico de memória e a exibe formatada para o usuário.

3.2 DIAGRAMA DE FLUXO

O fluxo de interação entre os componentes locais pode ser visualizado no Quadro 1.

Quadro 1 – Diagrama de Fluxo da Aplicação



Fonte:Victor Bellé (2025).

3.3.1 Prompt de Sistema

O histórico de chat sempre começa com uma mensagem de role: "system", definida pela função `make_system_prompt()`. Este prompt instrui o modelo sobre sua persona e escopo, focando em objetividade e no domínio de artigos científicos.

3.3.2 Memória de Curto Prazo (Janela Deslizante)

Implementada na classe `ShortMemoryChat`. Esta classe mantém uma lista de mensagens. Quando o histórico ultrapassa $k=5$ interações (5 pares de usuário/assistente), as interações mais antigas são descartadas (FIFO - First-In, First-Out), mantendo apenas a prompt de sistema original e os k últimos pares.

3.3.3 Variáveis de Ambiente

O arquivo `.env` configura três variáveis principais:

- `OLLAMA_MODEL`: Define qual modelo o Ollama deve usar (ex: "llama3.1").
- `OLLAMA_BASE_URL`: Define a URL do servidor (padrão: "http://localhost:11434").
- `OPENAI_TEMPERATURE`: Reutilizado para definir a temperatura do modelo Ollama (padrão: 0.2), controlando a criatividade das respostas.

4 EXPLICAÇÃO DO CÓDIGO

4.1 ARQUIVO PRINCIPAL: chatbot.py

O projeto consiste em um único arquivo Python que contém toda a lógica. O código é estruturado em funções (como `ollama_chat` e `main`) e uma classe (`ShortMemoryChat`).

4.2 GERENCIAMENTO DA MEMÓRIA

A classe `ShortMemoryChat` é o componente central de gerenciamento de estado. O método `add` é responsável por anexar novas mensagens e, crucialmente, aplicar a poda da janela deslizante, como demonstrado no Quadro 2.

Quadro 2 – Trecho da Classe `ShortMemoryChat` (Método `add`)

```
def main() -> None:
    # ... (inicialização) ...

    # Teste de conexão com o Ollama
    try:
        requests.get(f"{base_url.rstrip('/')}/api/tags", timeout=5)
    except Exception:
        print(f"{Fore.RED}Não foi possível conectar ao Ollama...")
        sys.exit(1)

    memory = ShortMemoryChat(k=5)
    print(f"{Fore.CYAN}Chatbot pronto. ...{Style.RESET_ALL}")

    while True:
        user_input = input(f"{Fore.GREEN}Você:{Style.RESET_ALL} ").strip()

        if user_input.lower() in {"exit", "sair", "quit"}:
            break
        if user_input.lower() == "reset":
            memory.clear()
            continue
        # ... (lógica de chamada e impressão) ...
```

Fonte: Victor Bellé (2025).

4.3 COMUNICAÇÃO HTTP COM OLLAMA

A função `ollama_chat` (Quadro 3) encapsula a lógica de comunicação. Ela formata o payload JSON exigido pela API do Ollama e utiliza a biblioteca `requests` para fazer a chamada síncrona, aguardando até 300 segundos pela resposta.

Quadro 3 – Função `ollama_chat`

```
def ollama_chat(
    base_url: str,
    model: str,
    messages: List[Dict[str, str]],
    temperature: float = 0.2
) -> str:

    url = f"{base_url.rstrip('/')}/api/chat"
    payload = {
        "model": model,
        "messages": messages,
        "stream": False, # Espera a resposta completa
        "options": {"temperature": temperature},
    }
    resp = requests.post(url, json=payload, timeout=300)
    resp.raise_for_status() # Lança erro se a API falhar
    data = resp.json()

    # Extrai a resposta do JSON
    return (data.get("message") or {}).get("content", "") or \
        data.get("response", "")
```

Fonte: Victor Bellé (2025).

4.4 LOOP PRINCIPAL E INTERFACE (CLI)

A função `main` (Quadro 4) inicializa o sistema, testa a conexão com o Ollama e inicia o loop de interação com o usuário, tratando dos comandos especiais `reset` (para limpar a memória) e `exit/sair`.

Quadro 4 – Trecho da Função `main`

```

def main() -> None:
    # ... (inicialização) ...

    # Teste de conexão com o Ollama
    try:
        requests.get(f"{base_url.rstrip('/')}/api/tags", timeout=5)
    except Exception:
        print(f"{Fore.RED}Não foi possível conectar ao Ollama...")
        sys.exit(1)

    memory = ShortMemoryChat(k=5)
    print(f"{Fore.CYAN}Chatbot pronto. ...{Style.RESET_ALL}")

    while True:
        user_input = input(f"{Fore.GREEN}Você:{Style.RESET_ALL} ").strip()

        if user_input.lower() in {"exit", "sair", "quit"}:
            break
        if user_input.lower() == "reset":
            memory.clear()
            continue
        # ... (lógica de chamada e impressão) ...

```

Fonte: Victor Bellé (2025).

5 INSTRUÇÕES DE EXECUÇÃO

Para executar o projeto, siga os passos abaixo:

1. Instale o Ollama para seu sistema operacional.
2. Baixe o Modelo: Abra o terminal e execute `ollama pull llama3.1` (ou o modelo que desejar).
3. Clone o Repositório: (Comando `git clone ...` se aplicável) e navegue para a pasta.
4. Crie o Ambiente Virtual: `python -m venv venv`
5. Ative o Ambiente:
 - Windows (PowerShell): `.\venv\Scripts\activate`
 - macOS/Linux: `source venv/bin/activate`
6. Instale as Dependências: `pip install -r requirements.txt`
7. Configure o `.env`: Crie um arquivo `.env` na raiz do projeto com o conteúdo (ajuste o modelo se necessário):


```

OLLAMA_MODEL="llama3.1" OLLAMA_BASE_URL="http://localhost:11434"
OPENAI_TEMPERATURE="0.2"
            
```

8. Execute o Chatbot: `python chatbot.py` (Nota: Se o arquivo estiver dentro de uma pasta app, o comando seria `python -m app.chatbot`)
9. Comandos no CLI:
 - Digite sua pergunta e pressione Enter.
 - `reset`: Limpa o histórico de conversa.
 - `exit` ou `sair`: Encerra o programa.

6 RESULTADOS E DEMONSTRAÇÃO

6.1 EXEMPLOS DE INTERAÇÃO

O chatbot se mostrou funcional e capaz de manter conversas curtas e contextuais sobre o domínio proposto.

Você: Me de dicas para escrever um artigo científico

Bot: Que ótimo desafio! Aqui estão algumas dicas básicas para escrever um artigo científico:

Estrutura do Artigo

1. **Introdução:** Apresente o problema ou questão que você está investigando, contextualizando-a e explicando por que ela é importante.
2. **Métodos:** Descreva como você realizou a pesquisa, incluindo os métodos utilizados para coletar dados e analisá-los.
3. **Resultados:** Apresente os resultados da sua pesquisa, de forma clara e objetiva.
4. **Discussão:** Analise os resultados e discuta suas implicações, relacionando-os à literatura existente.
5. **Conclusão:** Resuma as principais contribuições do seu artigo e sugira direções para futuras pesquisas.

Dicas Gerais

1. **Clareza e objetividade:** Use linguagem clara e objetiva, evitando jargões e termos técnicos desnecessários.
2. **Organização lógica:** Organize o artigo de forma lógica e fácil de seguir.
3. **Referências adequadas:** Cite as fontes utilizadas corretamente, seguindo um estilo de referência específico (por exemplo, APA, MLA ou Chicago).
4. **Gráficos e tabelas:** Use gráficos e tabelas para ilustrar os dados e facilitar a compreensão.
5. **Revisão crítica:** Peça feedback de colegas e especialistas antes de submeter o artigo para publicação.

Métricas

1. **Índice de citação (h-índice):** Mede a frequência com que seu artigo é citado por outros.
2. **Fator de impacto:** Avalia a influência do periódico em que o artigo foi publicado.
3. **Taxa de aceitação:** Mede a probabilidade de um artigo ser aceito para publicação.

Outras Dicas

1. **Seja objetivo e preciso:** Evite generalizações e conclusões precipitadas.
2. **Use linguagem científica:** Use termos técnicos e conceitos específicos da área de estudo.
3. **Inclua imagens e gráficos:** Ilustre os dados e facilitar a compreensão.

Lembre-se de que essas são apenas dicas básicas, e o melhor caminho é sempre consultar as diretrizes do periódico em que você deseja publicar seu artigo. Você tem alguma pergunta específica ou gostaria de saber mais sobre algum desses pontos

6.2 OBSERVAÇÕES DE LATÊNCIA

A latência (tempo de resposta) foi um fator notável, explicado por três motivos principais:

- **Carregamento Inicial:** A primeira requisição após iniciar o Ollama pode demorar (10-30 segundos), pois o modelo precisa ser carregado na RAM/VRAM.
- **Hardware:** A geração da resposta é computacionalmente intensiva e depende da CPU ou (preferencialmente) GPU do usuário.
- **stream=False:** Como a requisição não utiliza streaming, o usuário não vê a resposta até que o modelo tenha gerado o texto completo, aumentando a percepção de lentidão.

7 LIMITAÇÕES E TRABALHOS FUTUROS

7.1 LIMITAÇÕES

- **Ausência de RAG:** O chatbot não pode responder sobre documentos específicos.
- **Dependência de Hardware:** O desempenho é um gargalo em máquinas sem GPU dedicada.
- **Latência Percebida:** O uso de `stream=False` torna a experiência do usuário menos interativa.
- **Memória Fixa:** A janela de `k=5` pode ser muito curta para contextos complexos.

7.2 TRABALHOS FUTUROS

- **Habilitar Streaming:** A melhoria mais imediata seria modificar `ollama_chat` para `stream: True` e processar a resposta linha por linha (como server-sent events). Isso exibiria a resposta token a token, melhorando a percepção de velocidade.
- **Introduzir RAG:** Integrar bibliotecas como PyMuPDF (para ler PDFs) e FAISS (para criar um banco de dados vetorial), permitindo que o chatbot responda perguntas sobre artigos específicos.
- **Interface Gráfica:** Substituir a CLI por uma interface web simples utilizando Streamlit ou Flask.
- **Avaliação Objetiva:** Implementar um conjunto de testes para avaliar a acurácia do modelo em diferentes cenários.

8 CONCLUSÃO

Este projeto demonstrou com sucesso a criação de um chatbot acadêmico funcional, operando inteiramente em hardware local através do ecossistema Ollama. O objetivo didático de interagir com uma LLM via requisições HTTP diretas e gerenciar um estado de conversa básico com memória de curto prazo foi plenamente alcançado. O código-fonte, focado em clareza e sem dependências complexas, prova a viabilidade de ferramentas de IA privadas, gratuitas e offline, removendo as barreiras de custo e privacidade associadas às APIs comerciais. Embora suas limitações (como a falta de RAG) sejam claras, a arquitetura estabelecida serve como uma base robusta para futuras expansões.

REFERÊNCIAS

DAY, Robert A. **Como escrever e publicar um trabalho científico**. 6. ed. São Paulo: Santos, 2005.

OLLAMA. Run large language models locally. [S.l.]: Ollama, 2025. Disponível em: <https://ollama.com/>. Acesso em: 10 nov. 2025.

PYTHON SOFTWARE FOUNDATION. Python Language Reference, version 3.10. Wilmington: PSF, 2025. Disponível em: <https://docs.python.org/3/>. Acesso em: 10 nov. 2025.

REITZ, Kenneth. Requests: HTTP for Humans. [S.l.]: Python Software Foundation, 2025. Disponível em: <https://requests.readthedocs.io/en/latest/>. Acesso em: 10 nov. 2025.